



ЛЕКЦИЯ 4 (ДИСЦИПЛИНА «СРЕДСТВА КОМПЬЮТЕРНОЙ ГРАФИКИ»)

Выполнила Ветрова Ольга
Авенировна, доцент каф. АСОИ и У

Как работает функция COMMAND

- Если командой предусмотрены аргументы, то их значения перечисляются. **Аргументами могут быть координаты точек, опции и др.** Аргументы перечисляются в той последовательности, в которой Вы задавали бы их, формируя команду для графического редактора в режиме диалога.
- **В качестве аргументов могут использоваться выражения *Visual LISP*.** При этом необходимо, чтобы значение, возвращаемое выражением, соответствовало тому значению, которого требует данная команда графического редактора.
- **В качестве аргументов могут быть заданы переменные**, если значения этих переменных были ранее определены в программе, или непосредственно заданы значения. Такие значения необходимо заключать в кавычки.
- **Слово *PAUSE* в качестве аргумента обозначает переключение в режим графического диалога**, т.е. после чтения *PAUSE* команда ожидает одного аргумента (опцию, точку и др.) с клавиатуры, от «мыши» и т.д., в точности так, как если бы команда выполнялась просто в графическом редакторе².

Полезные сведения о функции **COMMAND**

- в одной функции может вызываться не одна команда, а последовательность команд;
- (*COMMAND* “”) соответствует вводу пробела с клавиатуры;
- (*COMMAND*) равносильно нажатию клавиши *CTRL/C*.

Примеры использования функции COMMAND

Пример 1. COMMAND “LINE” “1,1” “10,10” “”)
; Эта функция строит линию по точкам 1,1 и 10,10

Пример 2. (COMMAND “CIRCLE” “2P” “10,10” “20,20”)
; Строится окружность по точкам на концах диаметра

Пример 3. (COMMAND “CIRCLE” “10,10” PAUSE “LINE” “10,10” “10,5”)
(COMMAND “LINE” “0,0” “210,0” “210,297” “0,297” “CLOSE”) .

В последнем примере строится окружность с центром в точке 10,10 и с радиусом, задаваемым пользователем,
затем строится ломаная линия: две первые точки ее задаются из программы,
а для задания последующих точек предполагаются действия пользователя, как при выполнении команды *LINE* в *NanoCAD*.

Обратим внимание на то, как вводятся координаты точки в функции COMMAND:

- Координаты X и Y отделяются запятой, и все эти записи заключаются в кавычки:
- *(COMMAND “CIRCLE” PC R).*
- В этой функции точка центра и радиус окружности заданы переменными.
- Такая строка не может быть выполнена отдельно, а только в составе программы, где перед данной функцией выполняются другие функции, определяющие конкретные значения PC и R .
- Такие программы будут рассмотрены ниже.

Если при выполнении функции какой-либо аргумент представлен переменной, то этой переменной должно быть ранее присвоено значение (число, строка текста, список и др.).

- **Основное средство для присвоения значений переменных – функция *SETQ*:**
- *(SETQ < переменная1 > < выражение1 > [< переменная2 > < выражение2 >]...).*
- Функция действует следующим образом: выполняются действия, задаваемые выражением 1, и результат, возвращаемый этим выражением, становится текущим значением переменной 1. Если имеются другие «пары», то с ними функция поступает аналогично. Функция *SETQ* возвращает значение последнего выражения.
- Функция *SETQ* может использоваться с любым количеством аргументов, которое должно быть обязательно четным и не менее двух. *Функция SETQ – основное средство для сохранения значений, возвращаемых другими выражениями.*

Для хранения данных пользователь может вводить свои символы (переменные), не совпадающие по написанию с зарезервированными (предопределенными) или ранее занятыми.

- **Создание новых переменных** осуществляется с помощью функции **SETQ**, например:
- **(SETQ r1 15.33 s24 9) .**
- Здесь вводятся переменные *r1* и *s24*, получающие соответственно, значения *15.33* (вещественное) и *9* (целое). При этом к той части оперативной памяти, которая отведена для текущего рисунка, добавляются участки, занимаемые переменными. Если переменной присвоить значение *NIL*, то такая переменная из памяти удаляется, и ее место освобождается для других операций *LISP*. Переменные могут использоваться в любых выражениях.

Примеры использования функции SETQ

- *(SETQ A 10.0) ; записывает 10 в A*
- *(SETQ B (+ 123)) ; записывает в B результат сложения 1, 2 и 3, т.е. 6*
- *(SETQ A 1 B 2 C 3) ; записывает 1 в A, 2 в B, 3 в C, возвращает значение 3*
- *(SETQ D (SETQ A 1 B 2 C 3)) ; записывает 3 в D.*
- Любую из этих функций можно набрать на клавиатуре в ответ на приглашение *Nano CAD*.

Для ввода значений переменных пользователем служат **функции типа GET.**

- К ним относятся:
- **GETINT** — ввод целого числа;
- **GETREAL** — ввод вещественного числа;
- **GETSTRING** — ввод строки текста;
- **GETPOINT** — ввод точки;
- **GETDIST** — ввод расстояния;
- **GETANGLE** — ввод угла и некоторые другие функции.

Работу функций ***GETINT***, ***GETREAL***, ***GETSTRING*** рассмотрим на примере функции ***GETINT***, которая в общем виде выглядит так:

- ***(GETINT [< подсказка >])***.
- Эта функция создает паузу в выполнении программы и ожидает, пока пользователь введет с клавиатуры целое число. Функция возвращает введенное число. Необязательный аргумент *< подсказка >* — это текстовая константа. Если этот аргумент присутствует, то во время паузы на экране будет высвечиваться текст подсказки.

Пример работы с функцией *GETINT*

- *(SETQ NUM (GETINT “Введите номер детали <1>: ”))* .
- Здесь функция *GETINT* выводит на экран запрос “Введите номер детали <1>: ”.
- Ее возвращаемым значением будет *NIL*, если пользователь ответит простым нажатием клавиши *<Enter>*, или введенное пользователем допустимое целое число (например, 21).
- В случае ввода пользователем недопустимого целого числа (например, -5) функция *GETINT* выдаст сообщение об ошибочном значении и будет ожидать допустимого варианта ввода.
- Аналогично записываются и действуют функции *GETREAL* и *GETSTRING*.

Запишем правило для функции GETPOINT: (GETPOINT [< точка >] [< подсказка >]).

- Функция создает паузу и ожидает ввода точки.
- Точка может быть введена путем указания на экране или путем набора ее координат на клавиатуре в виде X, Y. Функция возвращает координаты точки (в виде списка).
- Аргумент < подсказка > имеет тот же смысл, что и для функций GETINT.
- Если задан аргумент < точка >, то при перемещении курсора на экране будет отображаться «резиновая линия» от данной «опорной» точки до текущего положения курсора.

Функция (*GETDIST* [*< точка >*]/[*< подсказка >*]) создает паузу и ожидает ввода расстояния.

- Расстояние может быть указано тремя способами:
- как действительное число с клавиатуры;
- путем указания двух точек (будет определено расстояние между точками);
- путем указания второй точки при наличии аргумента *< точка >*.
- В двух последних случаях между первой точкой и текущим положением курсора рисуется «резиновая линия».
- Функция возвращает действительное число, равное введенному расстоянию. Нетрудно видеть, что первый способ задания расстояния для пользователя (и для программы) будет эквивалентен действию функции *GETREAL*.
- Аналогично записывается и действует функция *GETANGLE*, которая возвращает угол (в радианах) между прямой, заданной двумя точками, и осью *X*.

Программа 2

- *Программа 2. Построение окружности.*
- *(DEFUN OKR; Имя описываемой функции*
- *(SETQ PC (GETPOINT “\nВведите центр:”))*
- *(SETQ R (GETDIST PC “\nВведите радиус:”))*
- *(COMMAND “CIRCLE” PC R))*
- Таковую программу можно записать на диск, затем в любое время вызвать с диска и запустить на выполнение, набрав (OKR) в ответ на приглашение *Nano CAD*. Для вывода информации на экран можно использовать функции *(PRIN (C, T))* в различных модификациях.

22.10.2023

***ПРИМЕР ПРОГРАММЫ ДЛЯ ЛАБОРАТОРНОЙ РАБОТЫ № 6
«ВВОД И ВЫВОД РАЗЛИЧНЫХ ТИПОВ ДАННЫХ.
ПОСТРОЕНИЕ ПРОСТЫХ ГРАФИЧЕСКИХ ПРИМИТИВОВ С
ПОМОЩЬЮ ЛИСП-ПРОГРАММЫ»***

Раздел 2.1

```

(defun lsp2_1 ()
  (setq T1 (list 36 66))
  (setq T2 (list 64 84))
  (setq T3 (polar T2 84 56))
  (command "_LINE" T1 T2 T3 ""))
)

(defun lsp2_2 ()
  (setq PC (list 84 106))
  (setq R 86)
  (command "_CIRCLE" PC R))
)

(defun lsp2_3 ()
  (setq A 31)
  (setq B 21)
  (setq C 21)
  (setq K (* A B C))
  (print K))
)

(defun lsp2_4 ()
  (setq A (getint "\nA: "))
  (setq B (getint "\nB: "))
  (setq C (getint "\nC: "))
  (setq K (* A B C))
)

(defun lsp2_16 ()
  (lsp2_1)
  (lsp2_2)
  (lsp2_3)
  (lsp2_4)
)

```

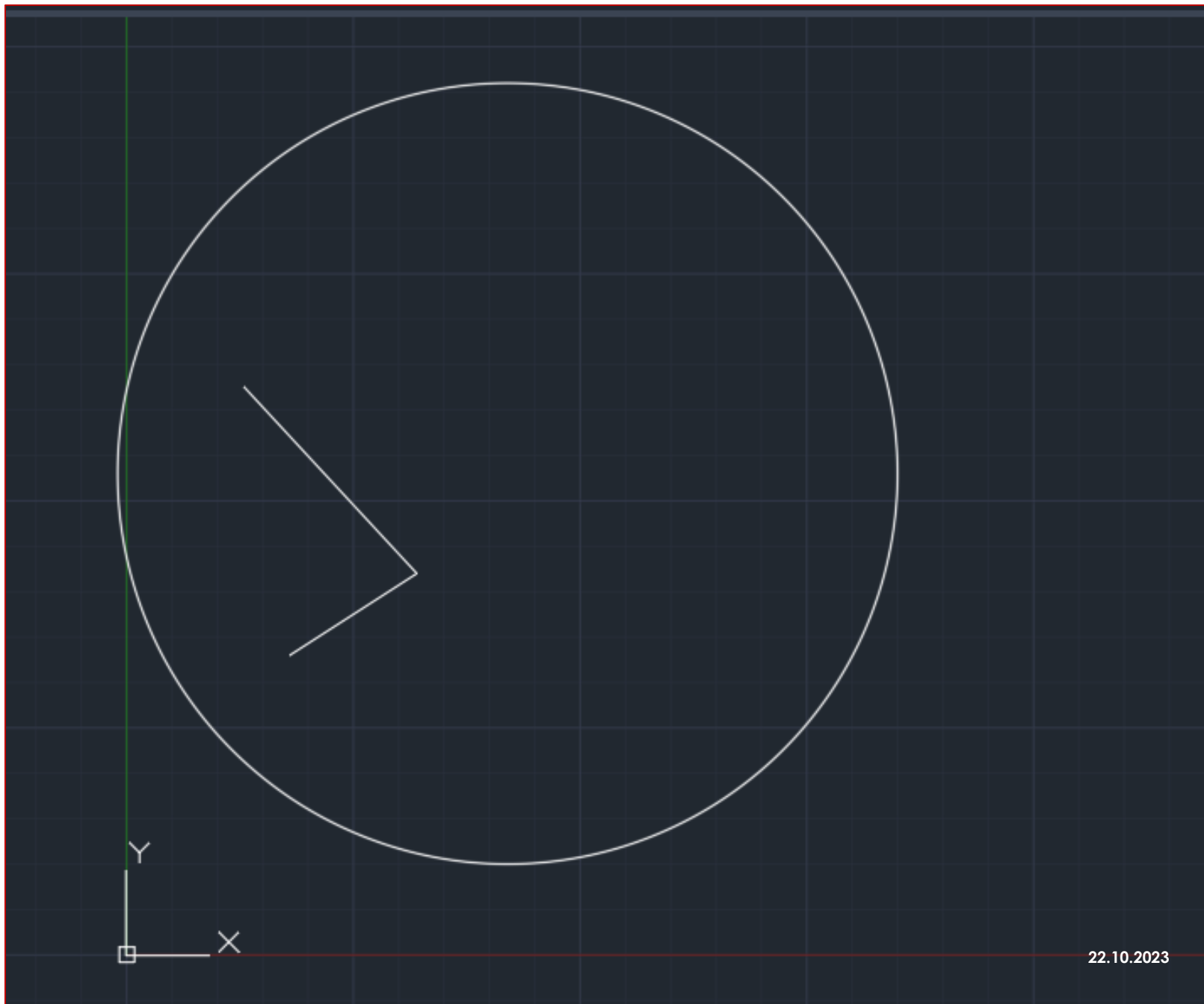
Пример программы для лабораторной работы № 6

*ПОСТРОЕНИЕ
ПРОСТЫХ
ГРАФИЧЕСКИХ
ПРИМИТИВОВ С
ПОМОЩЬЮ ЛISP-
ПРОГРАММЫ*

Результаты выполненной программы для лабораторной работы № 6

- Диалог с программой в командной строке графической системы

```
2_16.lsp successfully loaded.  
Command:  
Command:  
Command: (lsp2_16)  
_LINE  
Specify first point:  
Specify next point or [Undo]:  
Specify next point or [eXit/Undo]:  
Specify next point or [Close/eXit/Undo]:  
Command: _CIRCLE  
Specify center point for circle or [3P/2P/Ttr (tan tan radius)]:  
Specify radius of circle or [Diameter]: 86  
Command:  
13671  
A: 30  
B: 60  
C: 55  
99000  
Command: Regenerating model.
```



Вид рабочего пространства

**Изображение
построенных
графических
примитивов
для лабораторной
работы № 6**

22.10.2023

ГЕОМЕТРИЧЕСКИЕ ТОЧКИ В ПРОГРАММЕ НА VISUAL LISP

Раздел 3

Рассмотрим принцип построения **параметризованных изображений**. Для такого построения требуется задавать **геометрические объекты «внутри программы»**, т.е. без участия пользователя.

- Геометрическая точка в программе может быть задана пятью способами.
- **Первый** нам уже известен – **ввод точки пользователем через функцию GETPOINT.**
- **Второй** – **непосредственное указание координат точки в виде списка** из двух (двухмерная точка) или трех (трехмерная точка) элементов с помощью функции QUOTE.
- **Примеры:**
- `(QUOTE (1.0 1.0))` – точка с координатами 1,1; `'(1.0 1.0)` – аналогично;
- `'(1.0 2.0 3.0)` – точка в трехмерном пространстве с координатами $X=1$, $Y=2$, $Z=3$.

Третий способ — задание точки через другую точку, угол и расстояние с помощью функции *POLAR*:

(POLAR <точка> <угол> <расстояние>).

- Эта функция возвращает точку, находящуюся под углом *<угол>* и на расстоянии *<расстояние>* от заданной *<точки>*.
- Угол указывается в радианах против часовой стрелки.
- ***POLAR* — основная функция для внутрипрограммных геометрических построений.**
- Ее можно и удобно использовать практически во всех случаях.

Четвертый способ – определение точки пересечения двух отрезков с помощью функции *INTERS*:

(*INTERS* <точка1> <точка2> <точка3> <точка4> [<признак>]).

- Функция *INTERS* находит точку пересечения двух отрезков – **отрезка от <точки1> к <точке2> с отрезком от <точки3> к <точке4>** и возвращает найденную точку;
- **<признак>** – любое выражение *Visual LISP*.
- Если <признак> равен *NIL*, то будет определена точка пересечения линий бесконечной длины, наложенных на заданные отрезки.
- Если <признак> имеет любое значение, отличное от *NIL*, или если аргумент <признак> вообще отсутствует, то ищется точка пересечения только внутри отрезков. Если точка пересечения в этой ситуации не найдена, то функция *INTERS* возвращает *NIL*.

Пятый способ — с помощью функций *ANGLE* и *DISTANCE*.

- Функция *ANGLE* задается в виде правила:
- *(ANGLE <точка1> <точка2>)*.
- Функция *ANGLE* возвращает действительное число, равное углу в радианах, образованному прямой, проходящей через *<точку1>* и *<точку2>*, с осью *X*.
- Функция *(DISTANCE <точка1> <точка2>)* возвращает действительное число, равное расстоянию между *<точкой1>* и *<точкой2>*.

22.10.2023

ПРИНЦИП ПОСТРОЕНИЯ ПАРАМЕТРИЗОВАННЫХ ИЗОБРАЖЕНИЙ В ЛИСП-ПРОГРАММЕ

Раздел 4

Принцип построения параметризованных изображений иллюстрирует *программа 3*, позволяющая построить *квадрат с задаваемой стороной и вписанную в него окружность*.

Аналогично можно строить и более сложные изображения: для получения нового изображения с помощью программы пользователь задает новые значения параметров (точки, расстояния, числа и др.).

Из этих данных в программе могут быть получены другие данные с помощью математических функций, функции *POLAR* и т.п.

Полученные данные будут использованы в качестве аргументов команд *Nano CAD* посредством функции *COMMAND*.

22.10.2023

КВАДРАТ С ВПИСАННОЙ ОКРУЖНОСТЬЮ

Программа 3

Программа 3

- `(DEFUN C:QUADR ( )`
-
- `(SETQ P1 (GETPOINT`
`“\nНач. точка:”))`
- `(SETQ L (GETDIST`
`“\nДлина стороны:”))`
- ;заголовок описываемой функции
- ; Зададим начальную точку
— левый нижний угол квадрата:
- ; Зададим длину квадрата:

; определим три оставшиеся вершины квадрата:

(SETQ P2 (POLAR P1 0.0 L))

(SETQ P3 (POLAR P2 (/ PI 2) L)) ; угол = $PI/2$

(SETQ P4 (POLAR P3 PI L)); угол = PI

◦; Построим квадрат на экране:

◦(COMMAND "LINE" P1 P2 P3 P4 "C")

◦; (Здесь "C" задает опцию "Замкнуть")

*; Определим радиус окружности:
(SETQ R(/L 2)); радиус $R=L/2$*

*; Определим центр окружности PC:
(SETQ PR (POLAR P1 0.0 R)) ; PR - точка касания*

*; на нижней стороне
(SETQ PC (POLAR PR (/ PI 2) R))*

*; Построим окружность на экране:
(COMMAND "CIRCLE" PC R)
); Конец описания функции QUADR*

Спасибо за внимание!

